

Reporte

Proyecto 1: Uso de un protocolo existente (Model Context Protocol)

CC3067 Redes, Universidad del Valle de Guatemala

Jonathan Alejandro Díaz Tahuite, carné 23837

Repositorios:

- Anfitrión y servidor remoto: <https://github.com/Jonialen/mcp-chatbot>
- Servidor local propio: <https://github.com/Jonialen/brewops-mcp>

Qué se construyó

Un chatbot de consola que actúa como anfitrión MCP: lanza varios servidores del protocolo, reúne todas sus herramientas en una sola lista, se la entrega a un modelo, y ejecuta lo que el modelo pide.

El protocolo está implementado directamente sobre JSON-RPC 2.0, sin utilizar un SDK de MCP. Las únicas dependencias directas del proyecto son el cliente de Gemini para hablar con el modelo y un controlador de SQLite para guardar datos; ninguna toca un frame del protocolo.

Servidor	Transporte	Lenguaje	Autor	Herramientas
brewops	stdio	Go	propio	9
filesystem	stdio	TypeScript	Anthropic	14
git	stdio	Python	Anthropic	12
hotel	stdio	Python	compañero	9
rrhh	stdio	Python	compañero	6
netprobe	Streamable HTTP	Go, en contenedor	propio	3

Seis servidores conectados simultáneamente, 53 herramientas, tres lenguajes, cinco autores y los dos transportes que define el protocolo.

Los repositorios son [Jonialen/mcp-chatbot](https://github.com/Jonialen/mcp-chatbot), que contiene el anfitrión y el servidor remoto, y [Jonialen/brewops-mcp](https://github.com/Jonialen/brewops-mcp), el servidor local propio, publicado aparte para que otros estudiantes puedan instalarlo.

Especificación de los servidores desarrollados

Esta sección corresponde al punto 9 del enunciado.

De cada servidor propio se detalla su transporte, su ciclo de vida, sus métodos y los parámetros de cada herramienta que publica. Los esquemas y los frames que aparecen aquí fueron capturados de ejecuciones reales de los servidores, no redactados a mano.

Los servidores de compañeros que el anfitrión también consume quedan fuera de esta sección: están especificados por sus propios autores, en sus repositorios.

Fundamento común: JSON-RPC 2.0

MCP no define un formato de mensajes propio: utiliza JSON-RPC 2.0 en la capa de aplicación. Los tres tipos de mensaje comparten una misma forma en el cable y se distinguen únicamente por qué campos están presentes.

Tipo	method	id	Se responde
Petición	sí	sí	sí
Notificación	sí	no	nunca
Respuesta	no	sí	No aplica

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": { "name": "scale_recipe", "arguments": { } }
}
```

Un decodificador no puede saber de antemano qué tipo de mensaje ha llegado, por lo que la clasificación ocurre después de deserializar. Responder a una notificación es una violación del protocolo, no una cortesía.

Códigos de error Se utilizan los códigos estándar de JSON-RPC 2.0:

Código	Significado	Cuándo lo emiten estos servidores
-32700	Parse error	Un frame que no es JSON válido. La respuesta lleva <code>id: null</code> , porque no había <code>id</code> que leer.
-32600	Invalid Request	Reservado.
-32601	Method not found	Un método que el servidor no implementa.
-32602	Invalid params	Parámetros malformados, o una herramienta que el servidor nunca publicó.
-32603	Internal error	Fallo al serializar un resultado.

Distinción crítica: fallo de protocolo contra fallo de herramienta Esta distinción es la decisión de diseño más importante del protocolo y ambos servidores la respetan.

Situación	Cómo viaja
Método desconocido, parámetros inválidos, herramienta inexistente	Objeto error de JSON-RPC
Una herramienta se ejecutó y falló	Respuesta exitosa con <code>result.isError: true</code>

El motivo no es estilístico. El fallo de una herramienta está dirigido al modelo, para que lo lea y elija otro camino. Si viajara como error de protocolo, el cliente lo trataría como excepción y la conversación terminaría por un problema que el modelo podía haber resuelto.

```
{
  "jsonrpc": "2.0",
  "id": 3,
  "result": {
    "content": [{"type": "text", "text": "no coffee in the catalogue matches \"Sumatra\""}],
  }
}
```

```

    "isError": true
  }
}

```

Ciclo de vida El handshake son tres mensajes, no dos:

```

cliente → servidor    initialize                (petición)
cliente ← servidor    result: protocolVersion, ... (respuesta)
cliente → servidor    notifications/initialized    (notificación)

```

El tercer mensaje no es opcional: un servidor puede rechazar peticiones hasta recibirlo, y omitirlo produce una sesión que parece conectada y no responde.

Negociación de versión. Ambos servidores devuelven la versión que el cliente solicitó cuando pueden hablarla, en lugar de imponer la suya. Un cliente construido contra una revisión anterior habla una superficie de herramientas que no ha cambiado, y rechazarlo por el número costaría interoperabilidad sin ganar nada.

Métodos implementados

Método	Tipo	Descripción
initialize	petición	Abre la sesión y negocia versión y capacidades
notifications/initialized	notificación	El cliente confirma que está listo
tools/list	petición	Devuelve las herramientas publicadas y sus esquemas
tools/call	petición	Invoca una herramienta

Las superficies `resources/` y `prompts/` no se implementan. Ambos servidores declaran únicamente la capacidad `tools`.

BrewOps

Servidor de conocimiento de una cafetería de especialidad: catálogo, recetas, perfiles de tueste y registro de preparaciones.

Su razón de existir es que un modelo de lenguaje no tenga que inventar números. Ante la petición de escalar una receta, un modelo produce cifras que *parecen* correctas; una cafetería que necesita la misma taza dos veces no puede usar cifras que parecen correctas. Todas las herramientas calculan a partir de los registros del negocio.

Transporte y ejecución

Transporte	stdio, JSON-RPC delimitado por saltos de línea
Invocación	brewops [-db ruta]
Entrada	frames en stdin
Salida	frames en stdout, exclusivamente
Diagnóstico	stderr
Persistencia	SQLite (modernnc.org/sqlite , Go puro, sin cgo)

stdout transporta frames y nada más. Una sola línea ajena en ese flujo corrompe el stream para el cliente, razón por la cual todo diagnóstico va a stderr.

Instalación:

```
go install github.com/Jonialen/brewops-mcp@latest
```

Configuración en un anfitrión:

```
{
  "mcpServers": {
    "brewops": {
      "command": "/ruta/absoluta/a/brewops",
      "args": ["-db", "./brewops.db"]
    }
  }
}
```

En el primer arranque crea y siembra la base de datos, de modo que las herramientas tienen con qué responder de inmediato.

Herramientas

compare_roast_batches Compare two roast batches of the same coffee landmark by landmark, grade how significant each difference is, and name the change most likely to explain a difference in the cup. Use this when a roaster reports that a new batch of a familiar coffee is tasting different.

Parámetro	Tipo	Obligatorio	Descripción
batch_a	string	sí	The earlier batch label, used as the reference.
batch_b	string	sí	The later batch label, the one being questioned.
coffee	string	sí	Coffee name, whole or partial.

diagnose_extraction Compare a brew that was actually made against the shop’s recipe, report every variable that fell outside its window, and recommend the single change to make next. Use this whenever a barista describes a brew that did not come out as expected. The recommendation deliberately changes one variable at a time, because changing two makes the next cup impossible to interpret.

Parámetro	Tipo	Obligatorio	Descripción
coffee	string	sí	Coffee name, whole or partial.
dose_grams	number	sí	Grams of coffee used.

Parámetro	Tipo	Obligatorio	Descripción
grind_label	string	no	Grind setting used, if noted.
method	string	sí	Brewing method.
seconds	integer	sí	How long the extraction took, in seconds.
taste_notes	string	no	What the barista tasted, in their own words.
tds	number	no	Refractometer reading as a percentage, if one was taken. Optional; supplying it adds an extraction yield.
temp_c	number	no	Water temperature in Celsius, if it was measured.
water_grams	number	sí	Grams of water used.

get_coffee Look one coffee up by name and return everything the shop knows about it, including which methods it has recipes for. The name may be partial: ‘Guji’ finds ‘Ethiopia Guji Uraga’.

Parámetro	Tipo	Obligatorio	Descripción
name	string	sí	Coffee name, whole or partial.

get_recipe Return the shop’s recorded recipe for one coffee on one brewing method: ratio, water temperature, grind, bloom and the expected extraction window. Use this instead of recalling a general recipe, because these are the numbers this shop has settled on for this lot.

Parámetro	Tipo	Obligatorio	Descripción
coffee	string	sí	Coffee name, whole or partial.
method	string	sí	Brewing method, such as V60, Chemex, AeroPress, French Press or Espresso.

list_coffees List every coffee currently in the shop’s catalogue, with its origin, process, roast level, sensory notes, how many days it is off roast, and which brewing methods have a recipe. Use this to see what is actually available before recommending anything.

Sin parámetros.

list_roast_batches List the recorded roast batches for one coffee, with the landmarks of each: first crack, drop, development time and development time ratio. Use this before comparing two batches, to find out which labels exist.

Parámetro	Tipo	Obligatorio	Descripción
coffee	string	sí	Coffee name, whole or partial.

recommend_coffee Find the coffees in the shop's catalogue that best match a customer's request, optionally restricted to those with a recipe for a given method. Recommendations come only from what the shop actually has in stock.

Parámetro	Tipo	Obligatorio	Descripción
limit	integer	no	How many suggestions to return. Defaults to 3.
method	string	no	Restrict to coffees that have a recipe for this method. Optional.
notes	array<string>	sí	Flavour descriptors the customer asked for, such as floral, fruity, chocolate or bright.

record_extraction Save a brew that was made, so the shop builds a history of what has been tried on each lot. Use this after a barista reports a result they want kept.

Parámetro	Tipo	Obligatorio	Descripción
coffee	string	sí	Coffee name, whole or partial.
dose_grams	number	sí	Grams of coffee used.
grind_label	string	no	Grind setting used.
method	string	sí	Brewing method.
notes	string	no	What the barista tasted.
rating	integer	no	How the cup was rated, from 1 to 10.
seconds	integer	sí	How long the extraction took.
temp_c	number	no	Water temperature in Celsius.
water_grams	number	sí	Grams of water used.

scale_recipe Work a recipe out for a specific amount and return a full brew card: dose, water, bloom and the pour schedule with running totals. Give either the water you want to brew or the dose you have; the server derives the other from the shop's ratio. Always use this rather than calculating the dose yourself.

Parámetro	Tipo	Obligatorio	Descripción
coffee	string	sí	Coffee name, whole or partial.
dose_grams	number	no	Grams of coffee available. Give this or water_grams, not both.
method	string	sí	Brewing method.

Parámetro	Tipo	Obligatorio	Descripción
water_grams	number	no	Grams of water to brew. Give this or dose_grams, not both.

Ejemplo completo Petición:

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/call",
  "params": {
    "name": "scale_recipe",
    "arguments": { "coffee": "Guji", "method": "V60", "water_grams": 350 }
  }
}
```

Respuesta (contenido textual, con saltos de línea reales):

```
Ethiopia Guji Uruga on V60
dose          21.0 g
water         350 g
ratio         1:16.7
temperature   94 °C
grind         medium-fine (~650 µm)
pours
  bloom       41.9 g at 0:00 (total 42 g)
  first pour   154.1 g at 0:45 (total 196 g)
  second pour  154.0 g at 1:15 (total 350 g)
target time   2:45-3:10
rest          12 days off roast (peak)
```

El gramaje no lo produce el modelo: se deriva de la proporción 1:16.7 registrada por el negocio, y las etapas del vertido suman exactamente el total que mostrará la balanza.

netprobe

Servidor remoto de diagnóstico de red. Publica herramientas que resuelven nombres, abren conexiones TCP e intercambian peticiones HTTP, reportando lo que cada operación hizo realmente: las direcciones que un nombre resuelve, el tiempo que tomó un handshake, la versión de TLS y el cifrado que dos extremos negociaron.

Al ser el único componente genuinamente remoto del proyecto, lo que observa constituye evidencia sobre las capas inferiores al protocolo, en lugar de una descripción de ellas.

Transporte y endpoints

Transporte	Streamable HTTP (MCP 2025-06-18)
Endpoint MCP	POST /mcp

Salud	GET /health
Puerto	\$PORT, o :8080

Método HTTP	Comportamiento
POST /mcp	Recibe un frame JSON-RPC y responde con otro Termina la sesión indicada en Mcp-Session-Id 405. Este servidor no envía mensajes por iniciativa propia, y mantener una conexión abierta para mensajes que nunca llegan no es gratis
DELETE /mcp	
GET /mcp	

Cabeceras. La petición debe declarar ambos tipos aceptables, porque la respuesta puede ser un frame único o un flujo de eventos:

Content-Type: application/json

Accept: application/json, text/event-stream

Sesión. El servidor emite un identificador de sesión en la respuesta a initialize, y el cliente debe reenviarlo en toda petición posterior:

HTTP/1.1 200 OK

Content-Type: application/json

Mcp-Session-Id: 2697786776289542af033b2cd289be7a

A partir de initialize, el cliente incluye también la versión acordada:

Mcp-Session-Id: <identificador emitido>

MCP-Protocol-Version: 2025-06-18

Códigos de estado:

Estado	Cuándo
200	Hay una respuesta JSON-RPC en el cuerpo
202	El frame era una notificación; el cuerpo vacío es deliberado
400	Frame malformado

Herramientas

dns_lookup Resolve a hostname to its IPv4 and IPv6 addresses and report how long resolution took. Answers the question of what the name layer knows about a host.

Parámetro	Tipo	Obligatorio	Descripción
hostname	string	sí	The hostname to resolve, without a scheme or path.

http_probe Send a HEAD request to a URL and report the status, the response headers, the negotiated TLS version and cipher, and the time taken. Shows what the application and presentation layers agreed on.

Parámetro	Tipo	Obligatorio	Descripción
url	string	sí	An http or https URL.

tcp_probe Open a TCP connection to one or more ports on a host and report which accepted, which refused, and how long the handshake took. Measures reachability at the transport layer.

Parámetro	Tipo	Obligatorio	Descripción
host	string	sí	The host to connect to.
ports	array<integer>	sí	The TCP ports to try.

Ejemplo completo Handshake real capturado del servidor:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "capabilities": {
      "tools": {}
    },
    "protocolVersion": "2025-06-18",
    "serverInfo": {
      "name": "netprobe",
      "version": "1.0.0"
    }
  }
}
```

Invocación de una herramienta:

```
{
  "jsonrpc": "2.0",
  "id": 2,
  "method": "tools/call",
  "params": {
    "name": "dns_lookup",
    "arguments": { "hostname": "github.com" }
  }
}
```

Contenido de la respuesta:

```
github.com resolved in 132ms
IPv4: 140.82.112.4
IPv6: (none)
```

Restricciones de seguridad Una herramienta que realiza peticiones de red por cuenta de quien la invoca es una puerta hacia todo lo que el servidor alcanza. Desplegado en la nube, eso incluye el servicio de metadatos del proveedor y el resto de la red privada.

Antes de cualquier operación, el destino se resuelve y se rechaza si alguna de sus direcciones cae en un rango interno:

Rango	Ejemplo	Motivo
Loopback	127.0.0.1, ::1	El propio servidor
Privado	10.0.0.0/8, 192.168.0.0/16, 172.16.0.0/12	Red interna
Link-local	169.254.169.254	Metadatos de nube
No especificado	0.0.0.0	Sin destino

También se rechaza localhost por nombre y todo esquema que no sea http o https.

Resumen de interoperabilidad

El anfitrión desarrollado en este proyecto conecta simultáneamente servidores escritos en tres lenguajes distintos, por dos empresas distintas, sobre dos transportes distintos:

Servidor	Lenguaje	Origen	Transporte
filesystem	TypeScript	Anthropic (npm)	stdio
git	Python	Anthropic (PyPI)	stdio
brewops	Go	propio	stdio
netprobe	Go	propio	HTTP

El anfitrión no adapta nada para ninguno de ellos. Esa es exactamente la afirmación con la que el enunciado abre: que MCP hace que el desarrollo de la herramienta sea independiente del modelo que la consume.

Análisis por capas

Esta sección corresponde al punto 10 del enunciado.

Explica qué ocurre en las capas de enlace, red, transporte y aplicación cuando el anfitrión se comunica con un servidor MCP. Todos los números y descriptores que aparecen fueron medidos sobre el sistema en ejecución, no estimados.

El proyecto tiene dos transportes, y dan respuestas distintas

MCP define dos transportes, y ambos están implementados en este proyecto:

Transporte	Servidores	Dónde corre el servidor
stdio	brewops, filesystem, git, rrhh, hotel	Proceso hijo en la misma máquina

Transporte	Servidores	Dónde corre el servidor
Streamable HTTP	netprobe	Contenedor, accesible por red

La pregunta “qué sucede en cada capa” tiene respuestas radicalmente distintas para cada uno, y ese contraste es el hallazgo central de este análisis: uno de los transportes no usa la pila de red en absoluto.

Método La evidencia se obtuvo inspeccionando los procesos en ejecución:

- `/proc/<pid>/fd` para ver qué descriptores abre cada servidor.
- `ss -tnp` para contar sockets y conexiones TCP por proceso.
- `ip link` y `docker inspect` para la configuración de enlace y red.
- El registro de frames del anfitrión (`logs/mcp-*.log`) para los tamaños reales de los mensajes.
- El propio servidor `netprobe`, cuyas herramientas reportan lo que observan al resolver nombres, abrir conexiones TCP e intercambiar peticiones HTTP.

Transporte stdio: las capas que no existen

Un servidor stdio se ejecuta como proceso hijo del anfitrión. Los frames viajan por la entrada y salida estándar del proceso.

Inspeccionando `brewops` lanzado exactamente como lo lanza el anfitrión:

```
PID 187356
  fd 0 -> pipe:[580772]
  fd 1 -> pipe:[580774]
sockets abiertos:  0
conexiones de red: 0
```

Los descriptores 0 y 1 apuntan a pipes anónimos del kernel, identificados por su número de inodo. El proceso no tiene un solo socket abierto.

Esto significa que, para los cinco servidores locales del proyecto:

Capa	Qué ocurre
Enlace	Nada. No hay trama, no hay dirección MAC, no hay medio físico.
Red	Nada. No hay paquete IP, no hay direccionamiento, no hay enrutamiento.
Transporte	Nada en el sentido TCP/IP. No hay puerto, ni número de secuencia, ni control de flujo por ventana.
Aplicación	JSON-RPC 2.0 sobre un flujo de bytes.

Lo que sí existe, y hace el trabajo que en el caso remoto harían esas capas, es un pipe del kernel: un búfer en memoria del sistema operativo, con transferencia fiable y ordenada garantizada por el propio kernel y control de flujo por bloqueo cuando el búfer se llena.

Esta última propiedad no es teórica y tuvo una consecuencia concreta durante el desarrollo. Un servidor MCP escribe sus diagnósticos a `stderr`; si el anfitrión no lee ese pipe, el búfer del sistema operativo se llena, el servidor se bloquea en su siguiente escritura, y la sesión queda esperando una respuesta que nunca saldrá. El

síntoma aparenta ser un interbloqueo del protocolo y no lo es: es control de flujo de la capa que sustituye al transporte. El anfitrión dedica una goroutine exclusivamente a drenar `stderr` por esta razón.

Delimitación de mensajes Un pipe entrega un flujo de bytes sin fronteras: nada indica dónde termina un mensaje y empieza el siguiente. El transporte `stdio` de MCP resuelve esto delimitando cada frame con un salto de línea, lo que impone dos reglas:

1. Un frame no puede contener saltos de línea sin escapar.
2. `stdout` transporta frames y nada más. Una sola línea de log en ese flujo corrompe el stream para el cliente.

El cierre también es de esta capa Un servidor que no responde deja al anfitrión bloqueado escribiendo en un pipe cuyo búfer ya está lleno. Cerrar el transporte tiene que poder interrumpir esa escritura, porque de lo contrario apagar el chatbot depende de la buena voluntad de un proceso que precisamente dejó de colaborar. El transporte impone un plazo al cierre y mata el proceso hijo si expira.

Transporte Streamable HTTP: la pila completa

El servidor `netprobe` corre en un contenedor y se alcanza por red. Aquí sí participan las cuatro capas.

Capa de enlace Docker crea una topología de enlace virtual completa:

```
IP contenedor: 172.23.0.2
MAC:          1a:61:60:91:b1:93
gateway:      172.23.0.1
bridge:       br-cbee94fb37cd    MAC 92:a2:3c:52:ff:56
interfaces:   vethf9aa8f5@if2, veth59e8dc1@if2
MTU:          1500
```

El contenedor tiene su propia dirección MAC y se conecta mediante un par `veth`, dos interfaces virtuales unidas extremo a extremo, a un bridge que actúa como conmutador de capa 2. El bridge también tiene su propia MAC. Es una red Ethernet conmutada, implementada enteramente en software.

El MTU de 1500 bytes es el dato de esta capa con consecuencias medibles arriba, y se trata en la sección de transporte.

Capa de red

Direccionamiento	IPv4 privado, 172.23.0.0/16
Gateway	172.23.0.1, la interfaz del bridge en el host
Traducción	El host publica 0.0.0.0:8080 y lo redirige a 172.23.0.2:8080

La publicación de puertos de Docker es NAT: el contenedor no es directamente alcanzable desde fuera, y la traducción de direcciones ocurre en el host. Es la misma mecánica que cualquier servicio en la nube usa para exponer un contenedor con dirección privada al Internet público.

Cuando `netprobe` resuelve un nombre, esta capa se vuelve visible en la salida de la herramienta:

```
github.com resolved in 132ms
  IPv4: 140.82.112.4
  IPv6: (none)
```

Capa de transporte TCP, y el servidor escucha explícitamente:

```
LISTEN 0 4096 0.0.0.0:8080 0.0.0.0:*
```

La cola de conexiones pendientes es de 4096. La herramienta `tcp_probe` mide el establecimiento de conexión directamente:

```
TCP probe of github.com
  443    open    (70ms, local 172.23.0.2:41288)
```

Los 70 ms son el tiempo del three-way handshake completo, y el puerto local efímero 41288 es el que el kernel asignó a ese extremo de la conexión.

Segmentación, medida sobre frames reales. El registro del anfitrión muestra la distribución de tamaños de los mensajes MCP:

```
19 frames | mediana 204 B | máximo 13,077 B
```

El máximo es la respuesta a `tools/list` del servidor de filesystem, que publica 14 herramientas con sus esquemas JSON Schema completos. Con un MTU de 1500 bytes, el MSS resultante es de 1460 bytes (1500 menos 20 de cabecera IP y 20 de cabecera TCP), de modo que ese frame requiere 10 segmentos TCP. El mensaje mediano de 204 bytes entra en uno solo.

Esto ilustra una diferencia real entre los dos transportes: por loopback el MTU es de 65,536 bytes, y ese mismo frame de 13 KB viaja en un único segmento.

Capa de aplicación Sobre TCP hay tres protocolos apilados, y la herramienta `http_probe` los reporta todos:

```
HEAD https://github.com
  status: 200 OK (207ms)
  protocol: HTTP/2.0
  tls: TLS 1.3, cipher TLS_AES_128_GCM_SHA256
  cert: github.com, expires 2026-09-30
```

Nivel	Protocolo	Función
Cifrado	TLS 1.3	Confidencialidad, integridad y autenticación del extremo mediante certificado
Transferencia	HTTP/1.1 o HTTP/2	Método, cabeceras, códigos de estado
Aplicación	MCP sobre JSON-RPC 2.0	Semántica de herramientas

En el modelo OSI, TLS ocuparía las capas de sesión y presentación; en el modelo TCP/IP, que es el que la práctica sigue, todo esto es capa de aplicación.

El certificado del servidor es lo que permite verificar la identidad del extremo. Este detalle tuvo una consecuencia práctica en el proyecto: la imagen del contenedor no puede construirse sobre scratch, porque una imagen

sin certificados raíz falla toda conexión TLS con un error de autoridad no verificable que aparenta ser un fallo de red.

Qué hace MCP en la capa de aplicación

Esto es común a ambos transportes, y es donde vive el protocolo implementado.

Correlación de peticiones y respuestas JSON-RPC permite tener varias peticiones en vuelo simultáneamente. Cada una lleva un `id` que la respuesta debe devolver sin modificar, y el cliente mantiene una tabla que asocia cada `id` pendiente con quien espera esa respuesta.

Esta es la misma función que cumple el número de secuencia en TCP, resuelta de nuevo un nivel más arriba: el transporte garantiza que los bytes lleguen ordenados, pero no que las respuestas lleguen en el orden en que se pidieron. El servidor puede responder la tercera petición antes que la primera, y sin la correlación por `id` cada respuesta llegaría al solicitante equivocado.

Cabeceras propias del transporte HTTP El transporte HTTP añade dos cabeceras que el transporte `stdio` no necesita:

Cabecera	Función
<code>Mcp-Session-Id</code>	Emitida por el servidor en la respuesta a <code>initialize</code> y reenviada por el cliente en cada petición posterior. Sustituye lo que en <code>stdio</code> da la propia existencia del proceso hijo.
<code>MCP-Protocol-Version</code>	La versión acordada, enviada a partir de <code>initialize</code> .

La necesidad de la primera ilustra bien la diferencia entre transportes: en `stdio` la sesión es el proceso, y termina cuando el proceso termina. Sobre HTTP, que no tiene estado, la sesión debe construirse explícitamente.

Terminación de sesión

Transporte	Cómo termina
<code>stdio</code>	El anfitrión cierra <code>stdin</code> ; el servidor ve EOF y termina
HTTP	El cliente envía <code>DELETE /mcp</code> con el identificador de sesión

En el segundo caso la notificación es necesaria porque una sesión que el servidor sigue creyendo abierta retiene recursos del otro lado indefinidamente. Ese `DELETE` también lleva su propio plazo: un servidor que no responde al cierre no puede impedir que el cliente termine.

Comparación

Capa	<code>stdio</code>	Streamable HTTP
Enlace	No aplica	Ethernet virtual, par <code>veth</code> sobre bridge, MTU 1500

Capa	stdio	Streamable HTTP
Red	No aplica	IPv4 privado con NAT, 172.23.0.2
Transporte	Pipe del kernel: fiable, ordenado, control de flujo por bloqueo	TCP: puertos, handshake de 70 ms medido, segmentación por MSS de 1460 B
Aplicación	JSON-RPC 2.0 delimitado por saltos de línea	TLS 1.3 más HTTP/2 más JSON-RPC 2.0
Sesión	El proceso hijo	Mcp-Session-Id explícito
Alcance	Misma máquina	Cualquier red

La conclusión de la comparación es que MCP define la misma semántica sobre dos pilas que no comparten nada por debajo de la capa de aplicación. El cliente JSON-RPC implementado en este proyecto no distingue una de otra: ambas implementan la misma interfaz de tres operaciones, escribir un frame, leer un frame y cerrar, y todo lo que está por encima es idéntico.

Conclusiones

Esta sección corresponde al punto 11 del enunciado.

El protocolo cumple lo que promete, y se puede comprobar. El enunciado abre señalando que cada empresa define su propia forma de integrar herramientas y que por eso no hay interoperabilidad. El anfitrión desarrollado conecta simultáneamente seis servidores escritos en tres lenguajes, Go, TypeScript y Python, por cinco autores distintos, sobre dos transportes distintos, y le entrega sus 53 herramientas a un modelo de Google. No se escribió una sola línea de adaptación para ninguno. Que los servidores oficiales de Anthropic funcionen con un modelo que no es de Anthropic es la demostración más directa del argumento.

Implementar el protocolo a mano enseña lo que un SDK esconde. La decisión de escribir JSON-RPC directamente, sin SDK de MCP, obligó a resolver problemas que de otro modo habrían quedado invisibles: la correlación de identificadores, la distinción entre un fallo de protocolo y un fallo de herramienta, el drenaje de `stderr` para no bloquear al servidor, la negociación de versión con servidores de revisiones anteriores, el plazo de cierre cuando el otro extremo deja de responder. Ninguno de estos aparece en la documentación como una advertencia destacada; todos aparecen al primer contacto con un servidor real.

La distinción más importante del protocolo es también la más fácil de implementar mal. Una herramienta que se ejecutó y falló no viaja como error de JSON-RPC, sino como respuesta exitosa con `isError`. El destinatario de ese fallo es el modelo, que puede leerlo y elegir otro camino; tratarlo como excepción termina la conversación por un problema que era recuperable. El primer servidor oficial contra el que se probó el cliente confirmó esta distinción en la primera llamada.

Las capas inferiores solo son observables cuando existen. El resultado más instructivo del análisis fue descubrir que cinco de los seis servidores no usan la pila de red en absoluto: cero sockets, cero conexiones, dos pipes del kernel. Un análisis por capas de esos servidores no es un análisis corto, es un análisis vacío. El valor pedagógico está justamente en el contraste con el servidor remoto, porque muestra qué trabajo desaparece, y quién lo hace en su lugar, cuando no hay red de por medio.

Las decisiones de infraestructura tienen consecuencias en la capa de aplicación. Elegir `scratch` como imagen base habría producido un contenedor funcional en todo salvo en las conexiones TLS, fallando con un error que aparenta ser de red y no lo es. Compilar sin `CGO_ENABLED=0` habría producido una imagen que construye correctamente y falla al arrancar. Ninguno de los dos errores se manifiesta donde se origina.

Integrar el trabajo de otros es una prueba distinta a escribirlo. Los dos servidores de compañeros se agregaron pegando un bloque en un archivo de configuración, sin tocar una línea del anfitrión, que es exactamente lo que el formato declarativo promete. Pero el primero falló al arrancar por una ruta relativa que se resolvía contra el directorio de trabajo del proceso hijo y no contra el del anfitrión, y el segundo pareció no responder hasta que se notó que la prueba le cerraba la entrada estándar antes de que pudiera contestar. Ambos fallos estaban del lado del integrador, no del autor.

Lo que se haría distinto. La implementación cubre únicamente la superficie de herramientas del protocolo; `resources` y `prompts` quedaron fuera deliberadamente, y un anfitrión completo debería soportarlas. El transporte HTTP implementa la lectura de flujos de eventos pero el servidor propio siempre responde con un frame único, de modo que esa ruta está probada contra un servidor de prueba y no contra uno que realmente transmita por etapas. Finalmente, la gestión de contexto es acumulativa: una conversación larga terminará excediendo la ventana del modelo, y un anfitrión de producción necesitaría resumir o recortar el historial.